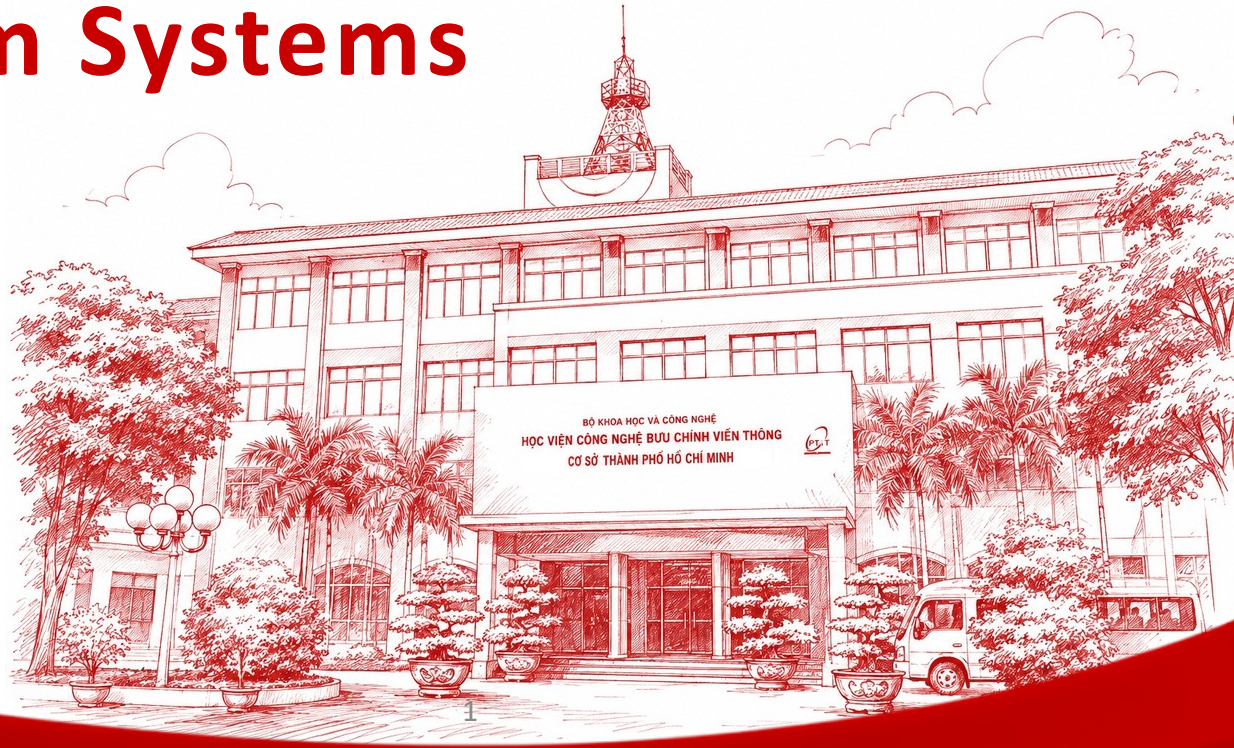
INT1405 - DISTRIBUTED SYSTEMS

LECTURE 08

Consistency Protocols, Replica Management, and Quorum Systems

Semester 1, 2026-2027

Lê Hà Thanh
lehathanh@ptithcm.edu.vn



Tanenbaum & van Steen (2nd Edition), Chapter 7 (Sections 7.4–7.5)

02. Learning Objectives & Operational Roadmap

Core Objectives

- **Master Replica Management**: Analyze server placement algorithms (Greedy selection) and content placement (Permanent, Server-Initiated, Client-Caches).
- **Evaluate Propagation Protocols**: Compare Push vs. Pull models and update payloads (Invalidation, Full State, Active Operations).
- **Understand Epidemic Gossip**: Master Anti-Entropy (Push/Pull), Rumor Spreading ($1/k$ probabilities), and Tombstones for deletions.
- **Analyze Primary & Quorum Protocols**: Contrast Remote/Local-Write Primary-Backup against Gifford's Majority Quorums ($N_R + N_W > N$).

Lecture Operational Structure

- Module 1: Replica Management & Content Placement (Slides 01–08)
- Module 2: Epidemic & Gossip-Based Protocols (Slides 09–16)
- Module 3: Primary-Based & Replicated-Write Protocols (Slides 17–24)
- Module 4: Cache Coherence, Web Caching & Advanced Case Studies (Slides 25–32)
- Module 5: Academic Assessment & Practical Gossip/Quorum Lab (Slides 33–40)

Content

- Module 1: Replica Management & Content Placement (Slides 01–08)
- Module 2: Epidemic & Gossip-Based Protocols (Slides 09–16)
- Module 3: Primary-Based & Replicated-Write Protocols (Slides 17–24)
- Module 4: Cache Coherence, Web Caching & Advanced Case Studies (Slides 25–32)
- Module 5: Academic Assessment & Practical Gossip/Quorum Lab (Slides 33–40)

03. The Replica Management Framework: Server vs. Content Placement

The Replica Management Problem

• **Primary Challenge**: Deciding where, when, and by whom replicas are instantiated and updated across a network topology.

• Two Fundamental Questions:

1. **Server Placement**: Where should replica servers be physically positioned?

2. **Content Placement**: Which data items should be allocated to which replica servers?

Placement Horizon & Lifecycle

• **Server Placement Horizon**: Long-term strategic infrastructure planning based on regional client request densities and network ISP topology.

• **Content Placement Horizon**: Dynamic, short-term data migration based on real-time traffic spikes and temporal access patterns.

04. Server Placement Algorithms: Greedy Topology Placement

The K-out-of-N Server Placement Problem

- **Goal**: Select K optimal replica server locations out of N candidate network locations to minimize overall client access latency.
- **NP-Hard Complexity**: Evaluating all N-choose-K combinations is computationally intractable for large networks.

Greedy Server Placement Heuristic

- **Step 1**: Compute average latency from all clients to all candidate server locations.
- **Step 2**: Select the single location that yields the lowest average client access latency.
- **Step 3**: Iteratively pick the next location that maximizes latency reduction for remaining underserved client clusters until K servers are placed.

05. Content Placement Taxonomy: Permanent, Server, and Client Replicas

Three-Tier Content Placement Classification

1. **Permanent Replicas**: Core static database mirrors configured manually by system administrators (e.g., mirrored databases).
2. **Server-Initiated Replicas**: Dynamic edge caches created automatically by servers to absorb regional traffic surges.
3. **Client-Initiated Replicas**: Local client caches managed entirely by web browsers, mobile proxies, or local applications.

Architectural Placement Scope

- **Permanent**: High availability and disaster recovery across core datacenters.
- **Server-Initiated**: Content Delivery Networks (CDNs) dynamically pushing content to edge nodes.
- **Client-Initiated**: Hiding network latency by serving repeated reads from local client memory.

06. Server-Initiated Replicas (Push Caching) & Threshold Mechanics

Dynamic Edge Replica Instantiation

- **Mechanism**: Origin servers track request origin frequencies per geographic region.
- **High Threshold Trip**: If regional access frequency for item x exceeds high threshold D_{high}, server dynamically instantiates a new replica near that region.
- **Low Threshold Trip**: If access drops below D_{low}, the replica is dropped to reclaim edge storage.

CDN Push Caching Benefits

- Absorbs regional flash crowds (e.g., breaking news or viral video traffic) near client edge.
- Prevents origin server CPU and network interface saturation.

07. Content Propagation Paradigms: Payload Selection Trade-offs

Three Update Payload Options

1. **Invalidation Notifications**: Send short tag notifying replicas data changed. Replicas discard stale data.

2. **Full State Transfer**: Transmit the entire updated data object payload to all replicas.

3. **Active Operations (State Machine)**: Transmit update code parameters (e.g., `Add(x, 5)`) rather than raw state.

Payload Selection Decision Matrix

- **Invalidations**: Best for low read-to-write ratios (frequent writes, rare reads); saves network bandwidth.
- **Full State**: Best for high read-to-write ratios (rare writes, frequent reads).
- **Active Operations**: Best for large data objects modified by tiny computational operations.

08. Push-Based vs. Pull-Based Propagation Protocols

Push-Based Propagation (Server-Driven)

- **Mechanism**: Updates are pushed proactively from master servers to replicas as soon as writes occur.
- **Characteristics**: High consistency; overhead incurred even if replicas are never read. Ideal for primary-secondary databases.

Pull-Based Propagation (Client-Driven)

- **Mechanism**: Replicas poll master servers explicitly when client reads occur or cache TTLs expire.
- **Characteristics**: Low network overhead when data is static; introduces latency on cache misses. Ideal for Web proxies.

Content

- Module 1: Replica Management & Content Placement (Slides 01–08)
- Module 2: Epidemic & Gossip-Based Protocols (Slides 09–16)
- Module 3: Primary-Based & Replicated-Write Protocols (Slides 17–24)
- Module 4: Cache Coherence, Web Caching & Advanced Case Studies (Slides 25–32)
- Module 5: Academic Assessment & Practical Gossip/Quorum Lab (Slides 33–40)

09. Epidemic Update Protocols: Gossip Architecture & Disease Models

The Epidemic Computing Philosophy

- **Concept**: Inspired by biological disease transmission models. Designed for massive peer-to-peer networks with no central coordinator.
- **Goal**: Rapidly converge all decentralized nodes to a consistent update state using localized, random message exchanges.

Key System Properties

- **Decentralized**: Zero reliance on single master nodes or global locks.
- **Fault Tolerant**: Resilient to severe node crashes and intermittent packet loss.
- **Scalability**: Logarithmic convergence time $O(\log N)$ rounds across thousands of nodes.

10. Node State Taxonomy in Epidemic Systems

Three Epidemic Node States

1. **Susceptible Node**: Node has not yet received a specific update and is eligible to be updated.

2. **Infective Node**: Node holds the update and is actively attempting to spread it to randomly selected peer nodes.

3. **Removed Node**: Node holds the update but has stopped attempting to spread it to peers.

State Transition Progression

- **Path**: Susceptible → Infective → Removed.
- **Goal**: Transition all Susceptible nodes to Infective/Removed quickly without flooding the network indefinitely.

11. Anti-Entropy Protocol Mechanics: Push, Pull, and Push-Pull

Anti-Entropy Synchronization Modes

- **Mechanism**: Node P randomly selects peer Q and exchanges full write sets to synchronize state.
- **Push Mode**: P sends its updates to Q. (Efficient when few nodes are infected).
- **Pull Mode**: Q fetches updates from P. (Efficient when most nodes are infected).

Push-Pull Hybrid Convergence

- **Push-Pull Mode**: P and Q exchange missing updates bidirectionally in a single round.
- **Convergence Rate**: Fastest epidemic synchronization mode; achieves exponential infection spread in $O(\log N)$ rounds.

12. Rumor Spreading (Gossiping) Protocols & Stopping Probabilities

Rumor Spreading Protocol Mechanics

- **Mechanism**: When node P receives a fresh update, it contacts random peer Q to push the update.
- **Already-Infected Check**: If Q was already infected, P suspects the rumor is widely known.
- **Stopping Probability**: P stops spreading the rumor with probability $1/k$ (transitioning to Removed state).

Trade-off Analysis

- **Advantage**: Extremely low network overhead compared to full Anti-Entropy.
- **Limitation**: Probability of residue (a small fraction of nodes may never receive the update).

13. Managing Deletions in Epidemic Systems: The Erasure Paradox

The Erasure Paradox

- **Problem**: If Replica A deletes an item by physically removing its record, a subsequent anti-entropy exchange with peer B will treat B's record as a missing update!
- **Consequence**: Replica A will re-insert the deleted item into its local database, making deletions impossible!

Root Cause Analysis

- Absence of data record is indistinguishable from an unreceived old write.
- Explicit delete operations must be recorded as positive update records.

14. Death Certificates (Tombstones) & Distributed Garbage Collection

Tombstone Execution Mechanics

- **Death Certificate**: Deletions write a special Tombstone record tagging the item as `DELETED` with a timestamp.
- **Epidemic Spread**: Tombstones spread across peer nodes via standard gossip updates, ensuring all replicas process the deletion.

Garbage Collection Cleanup

- **Tombstone Expiration**: Tombstones are retained for a cleanup interval $T_{cleanup}$ before being permanently purged.
- **Purge Condition**: $T_{cleanup}$ must exceed maximum anti-entropy sync delay to prevent deleted items from resurrected.

15. Case Study: Content Delivery Networks (CDNs) & Edge Caches

CDN Architecture Overview

- **Global Edge Network**: Deploying edge server clusters across worldwide ISPs (e.g., Cloudflare, Akamai).
- **Latency Masking**: Serving static assets and dynamic cached responses close to end users.

Edge Cache Update Strategies

- **Static Content**: Served via Pull caching with high Time-To-Live (TTL) expiry headers.
- **Dynamic Invalidations**: Origin servers send instant API-driven Push notifications to purge edge caches on write.

16. Anycast DNS & BGP Dynamic Edge Routing in Global CDNs

Anycast Routing Mechanics

- **Single IP, Multiple Nodes**: Multiple CDN edge clusters broadcast identical IP addresses using BGP.
- **Path Shortest Routing**: Internet routers direct client packets to the network-topologically closest edge cluster.

Failover & Traffic Load Balancing

- **Automatic Failover**: If an edge cluster crashes, BGP withdraws route, automatically shifting client traffic to nearest peer cluster.
- **Zero Client DNS Delay**: Rerouting occurs at router layer without waiting for client DNS cache expiry.

Content

- Module 1: Replica Management & Content Placement (Slides 01–08)
- Module 2: Epidemic & Gossip-Based Protocols (Slides 09–16)
- Module 3: Primary-Based & Replicated-Write Protocols (Slides 17–24)
- Module 4: Cache Coherence, Web Caching & Advanced Case Studies (Slides 25–32)
- Module 5: Academic Assessment & Practical Gossip/Quorum Lab (Slides 33–40)

17. Primary-Based Protocols: Remote-Write Primary-Backup

Remote-Write Architecture

• **Designated Primary**: A single master server coordinates all write operations; secondary backup servers hold read-only replicas.

Protocol Execution Sequence

1. Client sends write request to Primary P.
2. Primary P executes write locally and multicasts update to Backups B1..Bn.
3. Backups process update and return ACK to Primary P.
4. Primary P returns success to Client after receiving all ACKs.

• **Guarantees**: Strict Sequential Consistency; writes incur blocking network round-trip delays.

18. Remote-Write Primary-Backup Failure Modes & Latency

Primary Server Failure Hazard

- **Single Point of Failure**: If Primary crashes before broadcasting decision, backups are left in uncertain state.
- **Election Requirement**: Backups must execute election protocol (e.g., Bully/Ring/Paxos) to elect new Primary.

Performance Latency Bottleneck

- **Blocking Writes**:

  Write response time = Local Write + Max(Backup Network Latencies) + ACK aggregation.

- **WAN Scaling Limit**: High inter-datacenter latency degrades write throughput severely.

19. Primary-Based Protocols: Local-Write Primary-Backup Migration

Local-Write Migration Architecture

• Dynamic Primary Role: Primary server status migrates dynamically to whichever node currently needs to execute writes.

Protocol Execution Sequence

1. Client sends write request to local server replica. S1 (currently a backup).

2. Server S1 executes primary migration protocol: requests primary status and fetches latest state. from old Primary S0.

3. Server S1 becomes New Primary, executes write locally, and pushes updates asynchronously to secondary backups.

20. Primary Migration Mechanics for Mobile & Disconnected Workers

Mobile Worker Operating Model

- **Disconnected Work**: Mobile laptop user fetches primary status for key files before disconnecting from network.
- **Local Primary Execution**: Laptop executes writes locally as Primary without incurring network latency.

Reconnection Synchronization

- Upon reconnecting, mobile Primary pushes update log batch to master datacenter backups.
- **Use Case**: Distributed file systems and field mobile sales applications.

21. Replicated-Write Protocols: Active Replication & SMR

Active Replication Concept

- **Direct Multicast**: Writes are transmitted directly to all replicas simultaneously, rather than passing through a single primary server.

State Machine Replication (SMR)

- **Deterministic State Machines**: Each replica hosts a deterministic state machine.
- **Totally-Ordered Multicast**: Updates must be delivered to all replicas in the exact same order (e.g., Lamport timestamps or Paxos).

22. Totally-Ordered Multicast & Determinism Constraints in SMR

Requirement 1: Totally-Ordered Multicast

- All replicas must receive and apply incoming operations in the identical sequence.
- Enforced via logical consensus engines or central sequencer nodes.

Requirement 2: State Machine Determinism

- State transitions must depend ONLY on previous state and input parameters.
- **Forbidden Actions**: Reading system local time clocks, generating pseudo-random numbers, or inspecting un-ordered thread IDs.

23. Quorum-Based Protocols: Gifford's Weighted Majority Voting

Gifford's Quorum Concept (1979)

- **Quorum Voting**: Clients must secure permissions from a quorum (subset) of replicas before executing read or write operations.
- **Voting Allocation**: Total N replica servers. Each replica holds 1 vote.
- **Read Quorum (N_R)**: Number of replica votes required to execute a read.
- **Write Quorum (N_W)**: Number of replica votes required to execute a write.

Tunable Performance

- **Flexibility**: Allows tuning N_R and N_W to optimize for read-heavy or write-heavy application workloads.

24. Gifford's Quorum Rules: Read-Write & Write-Write Overlaps

Rule 1: Read-Write Overlap Rule

$$(N_R + N_W > N)$$

- **Requirement**:

$$N_R + N_W > N$$

- **Enforces**: The read quorum N_R and write quorum N_W must overlap by at least 1 replica holding the latest write version.

Rule 2: Write-Write Overlap Rule

$$(N_W > N/2)$$

- **Requirement**:

$$N_W > \frac{N}{2}$$

- **Enforces**: Two write quorums must overlap by at least 1 replica, preventing concurrent conflicting writes on split quorums.

Content

- Module 1: Replica Management & Content Placement (Slides 01–08)
- Module 2: Epidemic & Gossip-Based Protocols (Slides 09–16)
- Module 3: Primary-Based & Replicated-Write Protocols (Slides 17–24)
- Module 4: Cache Coherence, Web Caching & Advanced Case Studies (Slides 25–32)
- Module 5: Academic Assessment & Practical Gossip/Quorum Lab (Slides 33–40)

25. Cache-Coherence Protocols: Invalidate vs. Update Strategies

Cache-Invalidate Protocols

- **Mechanism**: Master server broadcasts short invalidation signals to client caches holding changed item.
- **Client Action**: Client marks local cache entry as invalid. Subsequent read triggers cache miss to fetch fresh data.
- **Best For**: High write volume with low read frequency.

Cache-Update Protocols

- **Mechanism**: Master server broadcasts updated data payload directly to active client caches.
- **Best For**: High read frequency; eliminates subsequent cache miss delays.

26. Write-Through vs. Write-Back (Write-Behind) Caching Mechanics

Write-Through Caching Strategy

• **Mechanism**: Client updates local cache, which immediately forwards write synchronously to master server store.

• **Properties**: High consistency;

Client write thread blocks until master store commits update.

Write-Back (Write-Behind) Caching Strategy

• **Mechanism**: Client updates local cache and returns success instantly. Cache flushes dirty memory to master in background batches.

• **Properties**: Ultra-fast write performance;

Risk of data loss if client device crashes prior to memory flush.

27. Web Proxy Caching Architecture & Conditional HTTP GET

Web Proxy Caches

• **Shared Client Caches**: Intercepting client HTTP requests at university or enterprise network boundaries.

Conditional HTTP GET Mechanics

• **Header Check**: Client sends HTTP GET with If-Modified-Since: `<timestamp>` header.

• **304 Not Modified**: If server object hasn't changed, returns light 304 code; proxy serves cached copy.

• **200 OK**: If object changed, returns code 200 with new data payload.

28. Hierarchical vs. Cooperative Web Caching & Cache Digests

Hierarchical Caching Topology

- **Tree Layout**: Regional proxies point to parent national proxies. Cache miss traverses up tree to origin.

Cooperative Caching & Cache Digests

- **Peer Interconnection**: Neighboring proxies query each other before routing cache misses to origin.
- **Cache Digests**: Proxies exchange compressed Bloom Filters summarizing local cache contents to reduce lookup traffic.

29. Real-World Case Study: Apache Cassandra Tunable Consistency

Cassandra Ring Architecture

• **Decentralized P2P Ring**: Key-value store using peer-to-peer ring topology and Dynamo-style partitioning.

Tunable Consistency Levels

• **ONE**: Query succeeds as soon as 1 replica responds.

• **QUORUM**: Query succeeds when a majority quorum $(floor(N/2) + 1)$ responds.

• **ALL**: Query succeeds only after all N replicas respond.

• **LOCAL_QUORUM**: Quorum enforced within client's local datacenter (ignores WAN delay).

30. Real-World Case Study: Google Bigtable & Chubby Paxos Locking

Bigtable Architecture

- **Distributed Structured Storage**: Manages petabytes across commodity servers.
- **Single-Row Consistency**: Tablet servers write log updates atomically to Colossus/GFS.

Chubby Lock Service

- **Coarse-Grained Locking**: Provides distributed locks and primary coordinator election.
- **Underlying Engine**: Uses Paxos active replication across 5 consensus replicas to guarantee strict consistency.

31. Trade-off Matrix across Consistency Protocol Families

Protocol Family	Write Latency	Fault Tolerance	Network Overhead
Remote Primary-Backup	High (Blocking WAN ACKs)	Tolerates N-1 backup failures	Moderate (Point-to-point)
Local Primary Migration	Low (Local write)	Requires migration protocol	Moderate (State transfer)
Active Replication (SMR)	Low (Multicast latency)	Tolerates minority failures	High (Total order multicast)
Gifford Weighted Quorum	Tunable (N_W quorum)	Tolerates $N - N_W$ write failures	Tunable (Quorum voting)
Epidemic Gossip	Zero (Async local)	Extremely high (P2P mesh)	Very Low (Background gossip)

Content

- Module 1: Replica Management & Content Placement (Slides 01–08)
- Module 2: Epidemic & Gossip-Based Protocols (Slides 09–16)
- Module 3: Primary-Based & Replicated-Write Protocols (Slides 17–24)
- Module 4: Cache Coherence, Web Caching & Advanced Case Studies (Slides 25–32)
- Module 5: Academic Assessment & Practical Gossip/Quorum Lab (Slides 33–40)

32. Protocol Selection Guidelines for Distributed Cloud Systems

Read-Heavy vs. High Availability Selection

• **High Read Throughput + Strong Consistency**: Select Gifford Quorum with $N_R = 1$, $N_W = N$ (or Primary-Backup with Read-Only Replicas).

• **High Availability + Partition Tolerance**: Select Gossip-based Epidemic Protocol or Cassandra Tunable QUORUM.

Deterministic Computation & Partition Selection

• **Deterministic Computation Execution**: Select Active Replication with Paxos/Raft State Machine Replication.

• **WAN Bandwidth Savings**: Select Invalidation Notifications or Active Operation parameter payloads.

33. Numerical Review Problems: Gifford Quorum Mathematics

Problem 1: Quorum Validation Test

• **Given**: Cluster N = 7 replicas. Proposed NR = 3, NW = 5.

Test this configuration against **Gifford's Read-Write Overlap Rule**.

• **Question A**: Satisfies Rule 1 $(N_R + N_W > N)$?

Calculation: 3 + 5 = 8 > 7. (PASS).

• **Question B**: Satisfies Rule 2 $\left(N_W > \frac{N}{2}\right)$?

Calculation: 5 > 3.5. (PASS).

Problem 2: Crash Tolerance Calculation

• **Question**: How many simultaneous server node crashes can this system tolerate during **Write operations** without halting?

• **Write Fault Tolerance** = $N - N_W$ 2 = 5 - 7 = server crashes tolerated.

• **Question**: How many simultaneous server node crashes can this system tolerate during **Read operations** without halting?

• **Read Fault Tolerance** = $N - N_R$ = 7 - 3 = 4 server crashes tolerated.

34. Numerical Review Problems: Epidemic Gossip Convergence Math

Problem 1: O(log N) Anti-Entropy Convergence

• **Given**: A 1,024-node peer-to-peer cluster using Push-Pull Anti-Entropy gossip synchronization.

• **Question**: Calculate the approximate number of gossip rounds R required for a fresh update to infect all 1,024 nodes across the network.

• **Formula**: Convergence rounds R approx. $\log_2(N)$.

• **Calculation**: $\log_2(1024) = 10$ rounds to infect all nodes.

Problem 2: Rumor Spreading Residue Probability

• **Given**: Rumor stopping probability 1/k where k = 2.

• **Result**: Over 98% of nodes are infected; under 2% residue remain susceptible.

• **Question**: Explain why Push-Pull mode achieves faster convergence than Push-only mode in late gossip rounds.

• **Answer:**

……

……

35. Conceptual & Analytical Review Questions

Self-Assessment Question 1

• Why is Active Replication required to be strictly deterministic, whereas Primary-Backup replication is not?

• **Model Answer**:
Active Replication sends operations to all nodes; non-deterministic functions cause replica divergence.
Primary-Backup executes on the primary and ships the resulting state.

Self-Assessment Question 2

• How do Death Certificates (Tombstones) solve the Erasure Paradox in epidemic gossip protocols?

• **Model Answer**:
Tombstones write an explicit DELETED tag that gossips to peers, preventing missing records from being mistaken for unreceived writes.

36. Practical Lab Assignment: Distributed Gossip & Quorum Simulator

Lab Overview & Assignment Goals

- **Assignment**: Distributed Gossip Protocol & Quorum Key-Value Store Simulator.
- **Language**: Python, Java, or Go.
- **Architecture**: Spawn 5 replica server processes (S1..S5) communicating via TCP/UDP sockets.

Required Features

1. Implement Gifford's Quorum voting (configurable N_R, N_W).
2. Implement Push-Pull Anti-Entropy gossip synchronization between replicas.
3. Implement Death Certificates (Tombstones) for deleted key-value entries.

37. Lab Implementation Details: Anti-Entropy Sync & Vector State

Socket & State Specifications

- **Key-Value Memory Structure**: key -> {value, version_number, is_tombstone}.
- **Periodic Gossip Thread**: Every 2 seconds, each replica picks a random peer socket and initiates Push-Pull anti-entropy exchange.

Quorum Query Protocol

- Client proxy sends read/write queries with specified N_R and N_W thresholds.
- Read quorums inspect version numbers across responding replicas and return the object with the highest version.

38. Lab Expected Console Trace Log & Evaluation Rubric

Sample Console Output Trace

```
[CLIENT] Write key='user1' val='Bob' (NW=3)
[S1] Write ACK Ver 1 | [S2] Write ACK Ver 1 | [S3] Write ACK Ver 1
[SYSTEM] Write Committed. S4 and S5 are stale (Ver 0).
[GOSSIP] S3 initiates Push-Pull Anti-Entropy with S4...
[S4] Memory State Updated -> key='user1' val='Bob' (Ver 1)
[CLIENT] Delete key='user1' -> Written as Tombstone (Ver 2)
[SYSTEM] All nodes successfully converged via Tombstone gossip.
```

Grading Rubric

- Gifford Quorum Voting Implementation (35%),
- Anti-Entropy Sync Logic (30%),
- Tombstone Handling (25%),
- Documentation (10%).

39. Summary Matrix of Consistency Protocols & System Dimensions

Protocol Type	Primary Architectural Mechanism	Best Use Case Scenario
Remote Primary-Backup	Centralized Primary routes writes to backups	Enterprise relational databases (PostgreSQL/SQL)
Local Primary Migration	Primary status shifts to active writer	Mobile & disconnected field worker applications
Active Replication (SMR)	Totally-ordered multicast to deterministic state machines	Mission-critical fault-tolerant flight control
Gifford Weighted Quorums	Overlapping Read/Write subset voting $(N_R + N_W > N)$	Tunable cloud key-value stores (Cassandra)
Epidemic Gossip	Randomized P2P anti-entropy exchange & tombstones	P2P networks, Dynamo-style distributed stores

40. Lecture 08 Key Takeaways & Transition to Course Capstone

Lecture 08 Takeaways

1. Replica & Content Management: Server placement balances long-term latency; content placement adapts dynamically.

2. Epidemic Scalability: Gossip protocols converge large-scale systems in $O(\log N)$ rounds without central locks.

3. Quorum Flexibility: Gifford's math $(N_R + N_W > N)$ enables tuning read vs. write performance per query.

4. Tombstones: Explicit death certificates are mandatory to process distributed deletions in decentralized stores.

Transition to Course Capstone & Exam Review

- **Next Topics**: Distributed Security, Web Services, and Enterprise Service-Oriented Architecture (SOA).
- **Action Item**: Complete the Gossip & Quorum Simulator Lab and review self-assessment problems.

Recommended Supplementary Resources

Gifford, D. K. (1979). "Weighted Voting for Replicated Data". *Proceedings of SOSP '79*, 150–162. (The seminal paper introducing Quorum-Based Voting and the overlap rules $(N_R + N_W > N)$ and $\left(N_W > \frac{N}{2}\right)$ for tuning read/write performance.)

Demers, A., et al. (1987). "Epidemic Algorithms for Replicated Database Maintenance". *Proceedings of PODC '87*, 1–12. (The classic paper introducing Anti-Entropy (Push, Pull, Push-Pull) and Rumor Spreading (gossip) protocols, along with Death Certificates (Tombstones) for distributed deletions.)

Schneider, F. B. (1990). "Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial". *ACM Computing Surveys*, 22(4), 299–319. (The definitive tutorial on Active Replication and State Machine Replication (SMR) using totally-ordered multicast.)

DeCandia, G., et al. (2007). "Dynamo: Amazon's Highly Available Key-Value Store". *Proceedings of SOSP '07*, 205–220. (Examines how Amazon combined vector clocks, gossip protocols, hint handoff, tombstones, and quorum voting (N, R, W) to build a highly available eventually consistent store.)

Lakshman, A., & Malik, P. (2010). "Cassandra: A Decentralized Structured Storage System". *ACM SIGOPS Operating Systems Review*, 44(2), 35–40. (Details Apache Cassandra's architecture and its implementation of Tunable Consistency (ONE, QUORUM, ALL, LOCAL_QUORUM).)